

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_6e519d4b-9e6\agent-a967fee0073416019.jsonl`
- SHA-256: `b623167d5fc47bf9463f4ba91697bd9595782e815d3cc091729839da5cf132db`
- Source modified: `2026-06-22T14:46:11+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_6e519d4b-9e6`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T14:44:22.149Z)

In C:/Users/avidu/Projects/bhi-cloudrun: find + read the existing cloud-serving TESTS (grep tests/ for CloudRunCompute, get_compute_backend, cloud_run, compute_target, fake client ? e.g. tests/test_compute*.py, test_worker*.py). Summarize what's already tested (the mocked dispatch) and the test patterns/fakes available. Also read deploy/cloudrun/Dockerfile + README.md: what's in the image, the entrypoint/CMD (is it `worker infer`?), the Miniconda + 'conda tos accept' lines (the P3a micromamba gap), how the image is built/run, and what weights/env it needs. Quote file:line. Flag whether any 'ingest a cloud result back into the API DB' code already exists (or is entirely absent = the item-1 gap).

2. assistant (2026-06-22T14:44:25.138Z)

I'll start by exploring the target repository structure, then dig into the tests and the Docker/deploy files.

3. user (2026-06-22T14:44:28.965Z)

You are not currently on a branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

```
git pull <remote> <branch>
```

---STATUS---

HEAD (no branch)

---LS---

```
total 219
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 ..
-rw-r--r-- 1 avidu 197609  611 Jun 22 20:13 .coveragerc
-rw-r--r-- 1 avidu 197609   88 Jun 22 20:13 .git
-rw-r--r-- 1 avidu 197609  947 Jun 22 20:13 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 .github
-rw-r--r-- 1 avidu 197609 1304 Jun 22 20:13 .gitignore
-rw-r--r-- 1 avidu 197609  525 Jun 22 20:13 CLAUDE.md
-rw-r--r-- 1 avidu 197609 35329 Jun 22 20:13 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 backend
-rw-r--r-- 1 avidu 197609  2735 Jun 22 20:13 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 deploy
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 docs
drwxr-xr-x 1 avidu 197609    0 Jun 22 20:13 eval_baselines
-rw-r--r-- 1 avidu 197609  2344 Jun 22 20:13 mypy.ini
-rw-r--r-- 1 avidu 197609  4525 Jun 22 20:13 pyproject.toml
-rw-r--r-- 1 avidu 197609   245 Jun 22 20:13 pytest.ini
```

```
-rw-r--r-- 1 avidu 197609 642 Jun 22 20:13 requirements.txt
-rw-r--r-- 1 avidu 197609 884 Jun 22 20:13 ruff.toml
-rwxr-xr-x 1 avidu 197609 2055 Jun 22 20:13 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5620 Jun 22 20:13 run_eval.py
-rw-r--r-- 1 avidu 197609 4356 Jun 22 20:13 run_regression.py
drwxr-xr-x 1 avidu 197609 0 Jun 22 20:13 scratch
drwxr-xr-x 1 avidu 197609 0 Jun 22 20:13 scripts
drwxr-xr-x 1 avidu 197609 0 Jun 22 20:13 tests
drwxr-xr-x 1 avidu 197609 0 Jun 22 20:13 tools
drwxr-xr-x 1 avidu 197609 0 Jun 22 20:13 training
```

4. user (2026-06-22T14:44:29.027Z)

C:\Users\avidu\Projects\bhi-cloudrun\tests\test_compute_backend.py

5. user (2026-06-22T14:44:29.034Z)

No files found

6. assistant (2026-06-22T14:44:31.491Z)

The repo is in detached HEAD state, so I'll note that and proceed read-only. Let me read the test file, find the deploy/cloudrun files, and locate the compute backend code.

7. user (2026-06-22T14:44:32.458Z)

```
1      """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3      All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
4      simulates the
5      worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
6      dispatch?bucket-poll
7      ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
8      ``compute_target=cloud_run``
9      selects a backend; everything else returns ``None`` (= run the worker's existing in-
10     process path).
11     """
12     import json
13     import os
14     import tempfile
15
16     from backend.compute import ComputeJobSpec, get_compute_backend
17     from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
18     from backend.config import AppConfig
19     from backend.infrastructure.execution_policy import ExecutionPolicy
20
21     # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
22     poll hits).
23     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
24
25     class _FakeRunClient(CloudRunJobClient):
26         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
27         local bucket
28         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
29         immediately."""
30
31         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
32             self.calls: list[dict] = []
33             self.video_id, self.segment_count, self.returncode = video_id, segment_count,
34             returncode
35
36         def run_job(self, job_name, argv, *, region, project=None):
37             self.calls.append({"job": job_name, "argv": list(argv), "region": region,
```

```

"project": project})
31         out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
32         done_uri = argv[argv.index("--done-uri") + 1]
33         outputs = os.path.join(out_uri, "outputs", self.video_id)
34         os.makedirs(outputs, exist_ok=True)
35         with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
36             json.dump({"video_id": self.video_id, "segment_count": self.segment_count,
37                       "status": "success"}, f)
38         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
39         with open(done_uri, "w", encoding="utf-8") as f:
40             json.dump({"video_id": self.video_id, "returncode": self.returncode,
41                       "segment_count": self.segment_count, "status": "success"}, f)
42         return "exec-123"
43
44
45     class _NoopRunClient(CloudRunJobClient):
46         """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
47         directly (a gs:// out_uri has no local dir to makedirs)."""
48
49         def __init__(self):
50             self.calls: list[dict] = []
51
52         def run_job(self, job_name, argv, *, region, project=None):
53             self.calls.append({"job": job_name, "argv": list(argv), "region": region,
"project": project})
54             return "exec"
55
56
57     # ----- factory (default-
OFF)
58
59     def _cfg(compute_target="", storage_backend="local"):
60         return AppConfig.model_validate({
61             "deployment": {"compute_target": compute_target},
62             "storage": {"backend": storage_backend},
63         })
64
65
66     def test_factory_is_none_for_local_targets():
67         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
68         assert get_compute_backend(_cfg("")) is None
69         assert get_compute_backend(_cfg("local_gpu")) is None
70         assert get_compute_backend(_cfg("cpu_mock")) is None
71
72
73     def test_factory_returns_cloud_run_for_cloud_target():
74         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
75         assert isinstance(backend, CloudRunCompute)
76
77
78     # ----- CloudRunCompute dispatch
+ poll
79
80     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
81         client = _FakeRunClient(video_id="abc123", segment_count=3)
82         backend = CloudRunCompute(run_client=client, job_name="rally-worker", region="asia-
south1",
83                                 project="proj", storage_config={}, policy=_FAST,
token="tok")
84         out = str(tmp_path / "bucket")
85         result = backend.run_infer(ComputeJobSpec(
86             video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"))
87

```

```

88     assert result.returncode == 0
89     assert result.video_id == "abc123"
90     assert result.outputs_uri == f"{out}/outputs/abc123/"
91     assert result.report.get("segment_count") == 3
92     # exactly one Cloud Run Job execution was triggered, in the right region/project
93     assert len(client.calls) == 1
94     assert client.calls[0]["job"] == "rally-worker" and client.calls[0]["region"] ==
"asia-south1"
95
96
97     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
98         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
99         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
100         client = _FakeRunClient()
101         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST,
token="tok")
102         backend.run_infer(ComputeJobSpec(
103             video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"), segmenter="wasb_hybrid",
104             config_overrides=("sport=tennis",))
105
106         argv = client.calls[0]["argv"]
107         assert argv[0] == "infer"
108         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
109         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
110         # the container-inline overrides come last (win) and the user override is carried
through
111         joined = " ".join(argv)
112         assert "deployment.compute_target=local_gpu" in joined
113         assert "indexing.detector_impl=native" in joined
114         assert "sport=tennis" in joined
115         # user override precedes the inline-forcing overrides in --set order
116         assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")
117
118
119     def test_unique_token_per_call_when_not_injected(tmp_path):
120         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
121         client = _FakeRunClient()
122         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
123         backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
124         backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
125         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
126         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
127         assert done1 != done2
128
129
130     def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
131         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
132         pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
133         identity passthrough). Mimic a cloud backend that rejects dst=None."""
134         from backend.compute import cloud_run as cr
135
136         captured = {}
137
138         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
139             captured["dst"] = dst
140             if dst is None:
141                 raise ValueError("cloud backend requires a dst path") # mirror

```

```

GCSStorage/S3Storage
142         with open(dst, "w", encoding="utf-8") as f:
143             f.write('{"video_id": "v", "returncode": 0}')
144         return dst
145
146     monkeypatch.setattr(cr, "fetch", fake_fetch)
147     backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
148     data = backend._read_json("gs://b/x.json")
149     assert captured["dst"] is not None # the fix: a real dst is always passed (no
silent {})
150     assert data == {"video_id": "v", "returncode": 0}
151
152
153     def test_cloud_run_polls_until_marker_appears(monkeypatch):
154         """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
155         polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
156         never exercises this; here exists() is False twice then True."""
157         from backend.compute import cloud_run as cr
158
159         exists_calls = {"n": 0}
160
161         class _FakeStore:
162             def exists(self, ref):
163                 exists_calls["n"] += 1
164                 return exists_calls["n"] >= 3 # absent for polls 1-2, present on poll 3
165
166         monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
167
168         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
169             if dst is None:
170                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-poll-"), "m.json")
171                 with open(dst, "w", encoding="utf-8") as f:
172                     f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
173                 return dst
174
175         monkeypatch.setattr(cr, "fetch", fake_fetch)
176         backend = cr.CloudRunCompute(
177             run_client=_NoopRunClient(), job_name="j", region="r", token="t",
178             policy=ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0,
max_wait_sec=5.0))
179         result = backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b"))
180         assert result.video_id == "vlate"
181         assert exists_calls["n"] >= 3 # >=1 absent poll occurred before the marker appeared
(loop ran)
182
183
184     # ----- worker wiring: dispatch branch +
done-marker
185
186     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
187         """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
188         from backend.compute.base import ComputeResult
189         from backend.worker import __main__ as wmain
190
191         seen = {}
192
193         class _FakeBackend:
194             def run_infer(self, spec):
195                 seen["spec"] = spec
196                 return ComputeResult(returncode=0, video_id="vidX",
outputs_uri="gs://b/outputs/vidX/")

```

```

197
198 monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
199 cfg = tmp_path / "c.json"
200 cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"gs"}}}')
201 rc = wmain.cmd_infer(wmain.build_parser().parse_args([
202     "infer", "--video-uri", "gs://b/videos/clip.mp4", "--out-uri", "gs://b",
203     "--config", str(cfg), "--segmenter", "wasb_hybrid"]))
204 assert rc == 0
205 assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
206 assert seen["spec"].segmenter == "wasb_hybrid"
207
208
209 def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
210     """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
211     remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
212     from backend.worker import __main__ as wmain
213
214     class _OK:
215         passed = True
216         validator_name = "fake"
217         message = "ok"
218
219         def model_dump(self):
220             return {"validator_name": "fake", "passed": True, "message": "ok"}
221
222     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
223                         lambda path, cfg: ([_OK()], {}))
224     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
225
226     monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
227                         lambda p: 30.0)
228
229     cfg = tmp_path / "c.json"
230     cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
231                                "output": {"output_dir": str(tmp_path / "o")}}))
232
233     video = tmp_path / "in.mp4"
234     video.write_bytes(b"FAKEVIDEO")
235     out = tmp_path / "bucket"
236     done = str(out / "_dispatch" / "tok.done")
237
238     rc = wmain.cmd_infer(wmain.build_parser().parse_args([
239         "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
240         "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid", "--done-uri",
done]))
241
242     assert rc == 0
243     marker = json.loads(open(done, encoding="utf-8").read())
244     assert marker["returncode"] == 0 and marker["status"] == "success"
245     assert marker["video_id"] and marker["segment_count"] >= 1
246
247
248 def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
249     """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
250     from backend.worker import __main__ as wmain
251
252     class _OK:
253         passed = True
254         validator_name = "fake"
255         message = "ok"
256
257         def model_dump(self):

```

```

255         return {"validator_name": "fake", "passed": True, "message": "ok"}
256
257     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
258                         lambda path, cfg: ([_OK()], {}))
259     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
260
261     monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
262                         lambda p: 30.0)
263     cfg = tmp_path / "c.json"
264     cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
265                                         "output": {"output_dir": str(tmp_path / "o")}}}))
266     video = tmp_path / "in.mp4"
267     video.write_bytes(b"FAKEVIDEO")
268     out = tmp_path / "bucket"
269     rc = wmain.cmd_infer(wmain.build_parser().parse_args([
270         "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
271         "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"]))
272     assert rc == 0
273     assert not (out / "_dispatch").exists() # no marker dir when --done-uri is omitted
274
275     # ----- Dockerfile structural guard
276     (no build)
277
278     def test_dockerfile_has_required_directives():
279         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
280         structurally:
281         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
282         worker entrypoint."""
283         from pathlib import Path
284
285         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
286         assert "FROM nvidia/cuda" in text # GPU base (CUDA)
287         assert "ffmpeg" in text # stitch/decode
288         assert "fonts-dejavu" in text # the watermark filter font
289         assert "torch==1.11.0+cu113" in text # the verified WASB stack
290         assert "WASB-SBDT" in text # the shuttle detector repo
291         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
292         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
293
294

```

8. user (2026-06-22T14:44:33.429Z)

No files found

9. user (2026-06-22T14:44:35.107Z)

Found 6 files

```

C:\Users\avidu\Projects\bhi-cloudrun\tests\test_storage.py
C:\Users\avidu\Projects\bhi-cloudrun\tests\test_startup_checks.py
C:\Users\avidu\Projects\bhi-cloudrun\tests\test_profile_parity.py
C:\Users\avidu\Projects\bhi-cloudrun\tests\test_config_deployment.py
C:\Users\avidu\Projects\bhi-cloudrun\tests\test_billing.py
C:\Users\avidu\Projects\bhi-cloudrun\tests\test_compute_backend.py

```

10. assistant (2026-06-22T14:44:38.278Z)

The deploy/cloudrun glob found nothing but the test references `deploy/cloudrun/Dockerfile`. Let me locate the actual deploy directory structure and the compute backend source, plus check the other test files that matched.

11. user (2026-06-22T14:44:39.976Z)

```

deploy/cloudrun/Dockerfile
deploy/cloudrun/README.md
deploy/digitalocean/bootstrap.sh
deploy/digitalocean/gpu_setup.sh
deploy/digitalocean/mount_scratch.sh
deploy/digitalocean/README.md
deploy/gcp/create_gpu_vm.sh
deploy/gcp/nightly_regression/cloudbuild.launcher.yaml
deploy/gcp/nightly_regression/create_nightly_bucket.sh
deploy/gcp/nightly_regression/Dockerfile.launcher
deploy/gcp/nightly_regression/launch.sh
deploy/gcp/nightly_regression/README.md
deploy/gcp/nightly_regression/register_scheduler.sh
deploy/gcp/nightly_regression/run_on_vm.sh
deploy/gcp/nightly_regression/setup_email.sh
deploy/gcp/ops-agent-startup.sh
deploy/gcp/provision.sh
deploy/gcp/README.md
deploy/gcp/teardown.sh
---COMPUTE SRC---
backend/compute/base.py
backend/compute/cloud_run.py
backend/compute/__init__.py
---WORKER---
backend/worker/__init__.py
backend/worker/__main__.py

```

12. user (2026-06-22T14:44:41.297Z)

```

88:                                     pricebook_version="v1", gpu_type="L4",
compute_backend="cloud_run",
93:     assert cost["gpu_type"] == "L4" and cost["owner_id"] == "u1" and cost["compute_backend"]
== "cloud_run"

```

13. assistant (2026-06-22T14:44:44.142Z)

The Dockerfile is at `deploy/cloudrun/Dockerfile`. The earlier glob just didn't surface it. Now let me read the Dockerfile, README, and the compute source files.

14. user (2026-06-22T14:44:44.637Z)

```

1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a

```



```

bucket fetch or a
16 # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18 FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20 ENV DEBIAN_FRONTEND=noninteractive \
21     PYTHONUNBUFFERED=1 \
22     PIP_NO_CACHE_DIR=1 \
23     CONDA_DIR=/opt/conda \
24     WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26 # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27 RUN apt-get update && apt-get install -y --no-install-recommends \
28     ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29     python3 python3-pip python3-venv \
30     && rm -rf /var/lib/apt/lists/*
31
32 # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33 RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34     && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35     && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36     && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37     && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38     && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40     torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41     && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43     hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45 # --- Main application env ---
46 WORKDIR /app
47 COPY pyproject.toml README.md ./
48 COPY backend ./backend
49 COPY training ./training
50 RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52 # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53 # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54 ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55     WASB_REPO_DIR=/opt/models/WASB-SBDT \
56     WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57     RALLY_DETECTOR_IMPL=native
58
59 # The dispatcher (CloudRunCompute) supplies the per-execution args:
60 # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61 ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

15. user (2026-06-22T14:44:45.577Z)

```

1 # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2

```

```

3      The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4      dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5      a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7      > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8      > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9      > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21     gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22     ```
23     *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26     ```bash
27     gcloud run jobs create rally-worker \
28     --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29     --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30     --max-retries 0 --task-timeout 3600 \
31     --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33     ```
34
35     ## 3. Wire the dispatcher (control plane)
36     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38     ```bash
39     export DEPLOYMENT_PROFILE=cloud-serving
40     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44     ```
45
46     ## 4. M7 acceptance (the owner runlog)
47     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test

```

```
(`ffmpeg -f null -`
50     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52     ## Gotchas
53     - The dispatched container must run inline ? `CloudRunCompute` forces
`compute_target=local_gpu`
54     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55     - Completion is a bucket marker (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57     - A failed Job currently writes no marker ? the dispatcher's bounded poll times
out
58     (`ExecutionPolicy.max_wait_sec`) and `_dispatch_remote` returns a clean rc 4 (not
a traceback).
59     A failure-marker (carrying the real non-zero rc, so the dispatcher distinguishes
failed from
60     *hung/cold-start-slow*) is a follow-up.
61     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
62
```

16. user (2026-06-22T14:44:45.659Z)

```
1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU Job and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. Dispatch a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7     2. Bucket-poll `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. Read the marker (it carries `video_id` + the worker's returncode) ? fetch
12        `outputs/<video_id>/report.json` ? `ComputeResult`.
13
14    The dispatched container is forced to run INLINE (`compute_target=local_gpu` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an injected client (`CloudRunJobClient`) so the whole
shim is unit-tested
18    with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-imports `google-cloud-
run` and is
19    owner-verified on the M7 real run.
20    """
21
22    from __future__ import annotations
23
24    import json
25    import logging
26    import os
27    import tempfile
28    import uuid
29    from abc import ABC, abstractmethod
30    from typing import Any, List, Optional
```

```

31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34     from backend.storage import fetch, parse_uri
35     from backend.storage import get_storage
36
37     LOG = logging.getLogger("compute.cloud_run")
38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency)."
72                 ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81

```

```

82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                     project: Optional[str] = None, storage_config: Optional[dict] = None,
87                     policy: ExecutionPolicy = DEFAULT_POLICY,
88                     token: Optional[str] = None,
89                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name
92             self._region = region
93             self._project = project
94             self._storage_config = storage_config or {}
95             self._policy = policy
96             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98             self._token = token
99             self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                             project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                     self._job_name, execution, done_uri)
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                                label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128             rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""

```

```

137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

17. assistant (2026-06-22T14:44:50.560Z)

Now let me read the worker `\_\_main\_\_.py` (the dispatch branch + done-marker writing), the compute base/factory, and check whether any "ingest a cloud result back into the API DB" code exists anywhere.

18. user (2026-06-22T14:44:51.554Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8     """
9
10    from __future__ import annotations
11
12    import os
13    from typing import Any, Optional
14
15    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18    __all__ = [
19        "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20    ]
21
22
23    def _storage_dict(config: Any) -> dict:
24        storage = getattr(config, "storage", None)
25        if storage is None:
26            return {}
27        if hasattr(storage, "model_dump"):

```

```

28         return storage.model_dump() # type: ignore[no-any-return]
29     return dict(storage) if isinstance(storage, dict) else {}
30
31
32     def get_compute_backend(
33         config: Any,
34         *,
35         run_client: Optional[Any] = None,
36         policy: ExecutionPolicy = DEFAULT_POLICY,
37         token: Optional[str] = None,
38     ) -> Optional[ComputeBackend]:
39         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40         in-process (the default-OFF path).
41
42         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46         """
47         target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48         if target != "cloud_run":
49             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57         client = run_client if run_client is not None else GoogleCloudRunJobClient()
58         return CloudRunCompute(
59             run_client=client, job_name=job_name, region=region, project=project,
60             storage_config=_storage_dict(config), policy=policy, token=token,
61         )
62

```

19. user (2026-06-22T14:44:51.992Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6     (cloud-serving) ? without the core ever branching on the profile (D1).
7
8     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9     non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Optional
18

```

```

19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str                # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str                  # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

20. user (2026-06-22T14:44:53.866Z)

```

29:from backend.compute import ComputeJobSpec, get_compute_backend
118:def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
121:    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.
138:        result.video_id, result.returncode, result.outputs_uri)
142:def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count: int,
144:    """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
remote
152:        storage.put(local, done_uri, config=storage_cfg)
153:        logger.info("[worker] wrote completion marker -> %s", done_uri)
155:        logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
164:    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
165:    # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through
to the
167:    # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.

```



```

168:     compute = get_compute_backend(config)
170:     return _dispatch_remote(compute, args)
242:     _write_report_json(video_id, segments, status, os.path.join(out_local, "report.json"))
255:     # M6: a remote dispatcher passes --done-uri; write a completion marker so its bucket-
poll
257:     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a failed
container
259:     if getattr(args, "done_uri", None):
260:         _write_done_marker(args.done_uri, video_id, 0, len(segments), status, storage_cfg,
scratch)
488:     pi.add_argument("--done-uri", default=None,

```

21. assistant (2026-06-22T14:44:57.065Z)

Let me read the worker `\_dispatch\_remote` function and the surrounding cmd_infer flow, then search the whole repo for any DB-ingest of cloud results.

22. user (2026-06-22T14:44:58.031Z)

```

115         return False # already logged at ERROR by enforce_startup_checks
116
117
118     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
119         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
120         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
122
123         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
124         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
125         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
126         from backend.infrastructure.execution_policy import PolicyTimeout
127
128         spec = ComputeJobSpec(
129             video_uri=args.video_uri, out_uri=args.out_uri,
130             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
131         )
132         try:
133             result = compute.run_infer(spec)
134         except PolicyTimeout as e:
135             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
136             return 4
137             logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
138                         result.video_id, result.returncode, result.outputs_uri)
139             return result.returncode
140
141
142     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
143                         status: str, storage_cfg: dict, scratch: str) -> None:
144         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
145         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
146         try:
147             marker = {"video_id": video_id, "returncode": returncode,
148                     "segment_count": segment_count, "status": status}
149             local = os.path.join(scratch, "_done.json")
150             with open(local, "w", encoding="utf-8") as f:

```

```

151         json.dump(marker, f)
152         storage.put(local, done_uri, config=storage_cfg)
153         logger.info("[worker] wrote completion marker -> %s", done_uri)
154     except Exception as e: # never fail a good run because the marker push hiccuped
155         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
156
157
158     def cmd_infer(args: argparse.Namespace) -> int:
159         config = load_config(args.config) if args.config else load_config()
160         _apply_overrides(config, args.set)
161         if not _run_startup_checks(config):
162             return 2
163
164         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
165         # process.
166         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
167         # through to the
168         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
169         # (the
170         # dispatcher forces compute_target=local_gpu), so it never recursively re-
171         # dispatches.
172         compute = get_compute_backend(config)
173         if compute is not None:
174             return _dispatch_remote(compute, args)
175
176         storage_cfg = config.storage.model_dump()
177
178         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
179         os.makedirs(scratch, exist_ok=True)
180         config.output.output_dir = scratch
181
182         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
183         # to scratch.
184         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
185         config=storage_cfg)
186         print(f"[worker] pulled {args.video_uri} -> {local_video}")
187
188         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
189         # byte-identical).
190         video_id = compute_video_id(local_video)
191
192         # 3. VALIDATE (same registry as the main CLI).
193         val_results, val_context = validator_registry.run_all_with_context(local_video,
194         config)
195         failures = [r for r in val_results if not r.passed]
196         db = Database(os.path.join(scratch, config.output.db_name))
197         db.add_video(video_id, local_video, "failed" if failures else "processed",

```

23. user (2026-06-22T14:45:00.006Z)

Found 22 files

```

C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\__main__.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\tools\export.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\ui\assets\index-GSXqL6iY.js
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\__init__.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\harvest.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\reporting\spec.yaml
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\yolo_hybrid.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\trajectory_hybrid.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\motion.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\personalization\serving.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\base.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\harness.py

```

```
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\regression.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\infrastructure\database.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\experiment.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud_run.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\billing.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\__init__.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\base.py
C:\Users\avidu\Projects\bhi-cloudrun\backend\api\static\index.html
```

24. assistant (2026-06-22T14:45:04.061Z)

Now let me check the API layer specifically ? that's where a "cloud result ? API DB" ingest would live. Let me look at the API process/job flow and whether it ever consumes a `ComputeResult` / `outputs\_uri` back into the DB.

25. user (2026-06-22T14:45:05.826Z)

```
backend/api/auth.py
backend/api/job_worker.py
backend/api/static/app.js
backend/api/static/index.html
backend/api/static/style.css
backend/api/uploads.py
backend/api/__init__.py
---
backend/api/job_worker.py:103:    ``config.billing.enabled``. When on, compute the job's cost
from ``result['usage']``
backend/api/job_worker.py:124:        gpu_type=usage.get("gpu_type"),
compute_backend=(result or {}).get("compute_backend"),
backend/api/job_worker.py:164:        logger.error(f"Could not compute next-due delay: {e}")
backend/api/uploads.py:23:from backend.utils.hashing import compute_video_id # canonical file-
identity hashing
backend/api/uploads.py:141:    video_id = compute_video_id(final_path)
backend/main.py:44:from backend.utils.hashing import compute_video_id # canonical file-identity
hashing
backend/main.py:342:    """Honest probe of what THIS box can do ? the first-run wizard renders
the compute step from
backend/main.py:369:        "compute_target": getattr(deployment, "compute_target", "") or
""
backend/main.py:459:    video_id = compute_video_id(local_video)
backend/main.py:1013:    video_id = compute_video_id(local_video)
```

26. assistant (2026-06-22T14:45:09.522Z)

The API layer (`backend/api/job\_worker.py`) handles jobs but doesn't reference `get\_compute\_backend`, `run\_infer`, `ComputeResult`, or `outputs\_uri`. Let me confirm how the API job worker runs inference ? whether it calls the worker/compute path at all, or only an in-process path.

27. user (2026-06-22T14:45:09.800Z)

```
1    """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3    Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4    The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5    that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6    re-claimed when due ? with no central scheduler (the DB is the clock).
7
8    The monkeypatch contract these functions rely on (and that tests depend on):
9    every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10   `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11   `backend.main` module object at call-time (`import backend.main as _main`), never as a
12   bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
```

```

13     ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15     inside the functions also avoids the circular import at module load (main.py imports
this
16     module; this module must not import main at import time).
17     """
18     import logging
19     import threading
20     import traceback
21     from concurrent.futures import ThreadPoolExecutor
22     from typing import Any, Dict, Optional
23
24     logger = logging.getLogger(__name__)
25
26     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29     # already parallelize their AI handoff internally via their own pools.
30     _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38         return _job_executor
39
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
49         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
50         """
51
52
53         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54             super().__init__(f"job parked for {delay_sec:.0f}s")
55             self.delay_sec = max(0.0, float(delay_sec))
56             self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71

```

```

72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(
87                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88             if result.get("video_id"):
89                 db_instance.set_job_video_id(job_id, result["video_id"])
90                 db_instance.mark_job_done(job_id, result)
91                 _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
92         except _main.JobWaiting as w:
93             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96         except Exception as e:
97             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105         persist it.
106
107         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
109         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111         With the placeholder pricebook the cost is 0 regardless.
112
113         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
114         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
115         import backend.main as _main
116
117         billing_cfg = getattr(_main.global_config, "billing", None)
118         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
119             return
120         try:
121             usage = (result or {}).get("usage") or {}
122             _main.db_instance.record_job_cost(
123                 job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
124                 gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
125                 owner_id=job.get("owner_id"), margin=billing_cfg.margin,
126             )

```

```

127         logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
128     except Exception as e: # never wedge a completed job on bookkeeping
129         logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)
130
131
132     def _drain_due_jobs() -> int:
133         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
134         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
135
136         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
137         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
138         a single-worker box keeps making progress with no central timer (the DB is the
clock).
139
140         Returns the number of jobs that reached a terminal/parked state this tick.
141
142         """
143         import backend.main as _main
144         ran = 0
145         while True:
146             job = _main.db_instance.claim_due_job()
147             if job is None:
148                 break
149             ran += 1
150             _main._run_claimed_job(job)
151             # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
152             # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
153             # on the next submit/drain. Best-effort ? never raises into the worker.
154             _main._schedule_next_drain_if_waiting()
155             return ran
156
157     def _schedule_next_drain_if_waiting() -> None:
158         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
159         self-scheduled job resumes with no further client submit. The drain itself re-checks
due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
160
161         import backend.main as _main
162         try:
163             delay = _main.db_instance.seconds_until_next_due()
164         except Exception as e: # pragma: no cover - defensive; never wedge the worker
165             logger.error(f"Could not compute next-due delay: {e}")
166             return
167         if delay is None:
168             return # nothing waiting
169         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
170         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
171
172     def _arm_wake_timer(delay_sec: float) -> None:
173         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
174         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
175
176         import backend.main as _main
177         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))

```

```

178         timer.daemon = True
179         timer.start()
180
181
182     #: A parked job further out than this is re-checked by a capped wake rather than one
long
183     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
184     _MAX_DRAIN_WAKE_SEC = 60.0
185

```

28. assistant (2026-06-22T14:45:13.638Z)

This confirms the key gap. The API job worker calls `\_main.\_execute\_processing(req, ...)` ? the in-process pipeline ? and never touches `get\_compute\_backend` / `CloudRunCompute`. Let me verify `\_execute\_processing` in `main.py` to confirm it's purely in-process and check the README's mention of "the API job path" to nail down the item-1 gap precisely.

29. user (2026-06-22T14:45:16.086Z)

```

369:         "compute_target": getattr(deployment, "compute_target", "") or "",
439: def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:

```

30. user (2026-06-22T14:45:16.512Z)

```

240         os.makedirs(out_local, exist_ok=True)
241         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
242         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
243
244         out_prefix = args.out_uri.rstrip("/")
245         pushed = []
246         for fn in sorted(os.listdir(out_local)):
247             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
248             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
249             pushed.append(uri)
250
251         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
252         for u in pushed:
253             print("    ", u)
254
255         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
256         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
257         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
258         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
259         if getattr(args, "done_uri", None):
260             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
261         return 0
262
263
264     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a

```

31. assistant (2026-06-22T14:45:20.316Z)

I have everything I need. The API `\_execute\_processing` at `main.py:439` has no compute-backend references ? confirming the cloud-result?DB ingest gap. Let me confirm the `\_execute\_processing` signature/body briefly to be certain it's in-process only and doesn't consume an `outputs\_uri`.

32. user (2026-06-22T14:45:20.791Z)

```
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
440         """The full hash ? validate ? segment ? report pipeline for one video.
441
442         This is the former synchronous /api/process body, unchanged in behavior, now run on
443         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
444         the sync endpoint used to return (UI compatibility); the per-video observability
445         report is still emitted in `finally:` and grafted as response["reports"].
446
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.
448         Raises on unexpected internal errors ? the caller records those as a job failure
449         (after this function has already restored the pre-run segments, see below).
450         """
451         notify = progress or (lambda stage: None)
452
453         notify("hashing")
454         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
455         # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
456         # original
457         # req.video_path (the source URI) is kept for the DB record + report; only the file-
458         # reading
459         # consumers (hash / validators / segmenter) use the resolved local path.
460         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
461         "storage", None))
462         video_id = compute_video_id(local_video)
463         was_reingest = db_instance.get_video(video_id) is not None
464
465         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
466         # report)
467         notify("validating")
468         logger.info(f"Running validations for {req.video_path}...")
469         val_results, val_context = registry.run_all_with_context(local_video, global_config)
470         failures = [r for r in val_results if not r.passed]
471
472         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
473         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
474         "motion")
475         seg_name = req.segmenter or default_seg
476         segmenter_actual = None
477         segmentation_ran = False
478         response: Optional[Dict[str, Any]] = None
479         try:
480             if failures:
481                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
482                 # status; it no longer destroys the video's prior good segments.
483                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
484                 for r in val_results])
```

33. assistant (2026-06-22T14:46:11.548Z)

Confirmed. `\_execute\_processing` runs the pipeline in-process and never calls
`get\_compute\_backend` / never reads a cloud `outputs\_uri` back into the DB. I have everything
needed.

Summary

****Repo state:**** `bhi-cloudrun` is in detached HEAD (`## HEAD (no branch)`), so I could not `git
pull --ff-only` (no branch to track). Read-only on the local checkout; may lag remote.

1. Cloud-serving TESTS ? what's already covered (all \$0, GPU-free, cloud-free)

****File:**** `C:\Users\avidu\Projects\bhi-cloudrun\tests\test\_compute\_backend.py` (290 lines, the single dedicated suite; `test\_billing.py:88,93` only asserts `compute\_backend="cloud\_run"` flows through the cost ledger).

The mocked dispatch flow proven (`CloudRunCompute` in `backend/compute/cloud\_run.py`):

- ****Factory default-OFF**** (`test\_factory\_is\_none\_for\_local\_targets:66`, `test\_factory\_returns\_cloud\_run\_for\_cloud\_target:73`): `get\_compute\_backend` returns `None` for `"/local\_gpu"/cpu\_mock`, a `CloudRunCompute` only for `compute\_target=cloud\_run`.
- ****Dispatch ? bucket-poll ? read**** (`test\_cloud\_run\_dispatch\_polls\_and\_returns\_result:80`): one `run\_job` call to the right region/project; `ComputeResult` carries `returncode`, `video\_id`, `outputs\_uri = {out}/outputs/{video\_id}/`, and `report.segment\_count`.
- ****argv construction**** (`test\_dispatch\_argv\_forces\_inline\_and\_carries\_overrides:97`): dispatched container is forced `INLINE` ? appends `deployment.compute\_target=local\_gpu` + `indexing.detector\_impl=native` AFTER user overrides (so user wins, and it never re-dispatches); `--done-uri`/`--video-uri`/`--out-uri` always present.
- ****Token uniqueness**** (`test\_unique\_token\_per\_call\_when\_not\_injected:119`): fresh uuid per dispatch so concurrent done-markers don't collide.
- ****Cloud-backend dst guard**** (`test\_read\_json\_passes\_dst\_for\_cloud\_backends:130`): `\_read\_json` must pass a real `dst` (GCS/S3 `fetch` raises on `dst=None`); regression guard against silent `{}`.
- ****Cold-start poll loop**** (`test\_cloud\_run\_polls\_until\_marker\_appears:153`): marker absent for 2 polls then present ? poll loops, doesn't resolve on poll 0.
- ****Worker wiring**** (`test\_cmd\_infer\_dispatches\_when\_cloud\_target:186`): `cmd\_infer` with `compute\_target=cloud\_run` hands off to the backend (no in-process run).
`test\_worker\_writes\_done\_marker\_on\_success:209` / `test\_worker\_no\_done\_uri\_is\_unchanged:245`: the container writes the done-marker only when `--done-uri` is passed (default-OFF otherwise).
- ****Dockerfile structural guard**** (`test\_dockerfile\_has\_required\_directives:277`): no build, just asserts the load-bearing directives exist (see §3).

****Fakes/patterns available to reuse:****

- `_FakeRunClient` (:21)`` ? subclasses `CloudRunJobClient`; on dispatch synchronously writes `outputs/<vid>/report.json` + the done-marker into a local tmp "bucket" so the poll resolves immediately. Ctor knobs: `video\_id`, `segment\_count`, `returncode`. Records every dispatch in `self.calls`.
- `_NoopRunClient` (:45)`` ? records the dispatch but writes nothing (for tests that monkeypatch the storage layer directly, e.g. a `gs://` out_uri with no local dir).
- `_FAST` ExecutionPolicy(poll_every_n_sec=0.0, max_wait_sec=5.0)` (:18)`` ? no-wait policy.
- `_cfg(compute_target, storage_backend)` (:59)`` ? `AppConfig.model\_validate` helper.
- Storage monkeypatch pattern: `monkeypatch.setattr(cr, "fetch", ...)` , `monkeypatch.setattr(cr, "get\_storage", ...)` with a `\_FakeStore.exists()` (:161).
- Worker-level fakes: stub validator (`\_OK` with `model\_dump`), `monkeypatch` of `validator\_registry.run\_all\_with\_context`, `emit\_indexer\_report`, and `trajectory\_hybrid.get\_video\_duration`; uses `detector\_impl=stub` + `skip\_ai\_handoff` for a deterministic GPU-free run.

The real client `GoogleCloudRunJobClient` (`cloud\_run.py:60`) lazy-imports `google-cloud-run` and is `# pragma: no cover` ? never exercised in CI (owner-verified on the M7 run only).

2. deploy/cloudrun/Dockerfile + README

****Dockerfile**** (`deploy/cloudrun/Dockerfile`, 62 lines):

- ****Base:**** `FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04` (:18) ? GPU/CUDA image, CI does NOT build it.
- ****Image contents:**** ffmpeg + `fonts-dejavu-core` (watermark) + git/curl/build-essential/python3 (:27-30); a separate conda `wasb` env (py3.8 + `torch==1.11.0+cu113` / `torchvision==0.12.0+cu113`, :39-40) cloned from <https://github.com/nttcom/WASB-SBDT.git> (:38) for the shuttle detector as a subprocess; the main app env is `pip install .` of this repo (:47-50).
- ****Entrypoint:**** `ENTRYPOINT ["python3", "-m", "backend.worker"]` (:61) ? args (`infer --video-uri ? --out-uri ? --done-uri ? --segmenter wasb\_hybrid --set ?`) are supplied per-execution by `CloudRunCompute`. So ****yes, it runs `worker infer`**** (the `infer` subcommand is injected by the dispatcher, not hard-coded as CMD).
- ****Weights/env needed:**** `WASB\_PYTHON=/opt/conda/envs/wasb/bin/python`,
`WASB\_REPO\_DIR=/opt/models/WASB-SBDT`, `WASB\_WEIGHTS=/opt/models/WASB-SBDT/pretrained\_weights/wasb\_badminton\_best.pth.tar`, `RALLY\_DETECTOR\_IMPL=native` (:54-57).

****Weights are NOT baked in**** (WASB-C3 ? academic provenance not cleared for commercial sharing, `:14-16`); staged at runtime via bucket fetch/mount, env points `\$WASB\_WEIGHTS` at them.

****The P3a micromamba/Miniconda gap**** ? the Dockerfile uses ****Miniconda****, not micromamba:
...

```
33 RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34     && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35     && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36     && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37     && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
...
```

The two `conda tos accept ? || true` lines (`:35-36`) swallow failures (the `|| true`), and they pull the Anaconda defaults channels ? exactly the Anaconda-ToS / channel friction that motivates a micromamba swap. This installs the full `Miniconda3-latest` distribution from `repo.anaconda.com` (defaults channels, subject to Anaconda's commercial ToS), not a conda-forge-only micromamba bootstrap.

****README**** (`deploy/cloudrun/README.md`): build via `gcloud builds submit --tag ? -f deploy/cloudrun/Dockerfile .` (\$1); create the GPU Job via `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi --max-retries 0 --task-timeout 3600` (\$2); wire the dispatcher with `DEPLOYMENT\_PROFILE=cloud-serving` + `CLOUD\_RUN\_JOB`/`CLOUD\_RUN\_REGION`/`GOOGLE\_CLOUD\_PROJECT` env (\$3). Status: dispatch shim + image are ****landed + mock-tested (M6)****; the real build+push+run on an L4 is the ****M7 owner step****. Notes a known gotcha: a ****failed**** Job writes ****no**** marker ? dispatcher's bounded poll times out ? clean ****rc 4**** (a failure-marker is a follow-up).

3. "Ingest a cloud result back into the API DB" ? ABSENT (the item-1 gap)

****No such code exists.**** The cloud path stops at returning a `ComputeResult` (returncode + `outputs\_uri` + `video\_id` + `report`); nothing reads those bucket artifacts (`outputs/<vid>/intervals.csv`, `report.json`) back into the API's SQLite DB.

- `backend/worker/\_\_main\_\_.py::dispatch\_remote` (`:118-139`) calls `compute.run\_infer(spec)` and ****just returns `result.returncode`**** ? it logs `video\_id`/`outputs\_uri` but never persists segments to a DB. (The non-dispatch inline path, by contrast, does write to `Database` at `:188`.)
- The ****API job worker**** `backend/api/job\_worker.py` is fully ****in-process and cloud-unaware****: `\_run\_claimed\_job` (`:72`) calls `\_main.\_execute\_processing(req, ?)` and `db\_instance.mark\_job\_done(job\_id, result)` ? it has ****no**** import/reference to `get\_compute\_backend`, `CloudRunCompute`, `run\_infer`, `ComputeResult`, or `outputs\_uri`. (`grep` of `backend/api` + `main.py` for those terms returns only the billing `compute_backend` string at `job_worker.py:124` and a capabilities-probe read of `compute_target` at `main.py:369`.)`
- `backend/main.py::\_execute\_processing` (`:439`) is the in-process `hash?validate?segment?report` pipeline; it `compute\_video\_id` locally and writes to `db\_instance` directly ? it never dispatches to a compute backend nor consumes a remote `outputs\_uri`.

So the only consumer of `compute\_target=cloud\_run` is the ****CLI**** `backend.worker infer` path (`cmd\_infer:168`). The hosted ****API**** (`/api/process` ? job worker ? `\_execute\_processing`) has ****zero**** cloud-dispatch wiring, and there is ****no**** "read the bucket result back into the API DB" code anywhere ? that ingest is entirely net-new (item-1 gap confirmed).

****Key paths:****

- `C:\Users\avidu\Projects\bhi-cloudrun\tests\test\_compute\_backend.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud\_run.py`,
`backend\compute\\_\_init\_\_.py`, `backend\compute\base.py`
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\worker\\_\_main\_\_.py` (dispatch `:118-170`, done-marker `:142-156`, inline DB write `:188`)
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\api\job\_worker.py` (`\_run\_claimed\_job:72`, `\_execute\_processing` call `:86`)
- `C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py::\_execute\_processing` (`:439`)

- `C:\Users\avidu\Projects\bhi-cloudrun\deploy\cloudrun\Dockerfile`, `deploy\cloudrun\README.md`